

FactCheck AI

System Architecture Design Document

Autonomous Multi-Agent Fact-Checking System using Locally Deployed LLMs and Live Web Retrieval

Document Type	Architecture Design Document
Version	1.0
Status	Draft — Phase 1
Date	May 2026
Author	Hirushikanth Vidyanandan
Student ID	CL/BSCSD/34/25 · ST20305792
Programme	BSc (Hons) Software Engineering
Institution	ICBT Campus, Colombo

CONFIDENTIAL · FOR ACADEMIC ASSESSMENT ONLY

Table of Contents

1. Document Information	4
1.1 Purpose.....	4
1.2 Intended Audience	4
1.3 Document Revision History	4
1.4 Related Documents	4
2. Executive Summary	5
3. System Overview	6
3.1 Problem Context.....	6
3.2 System Goals	6
3.3 Scope Boundaries	6
4. Architecture Overview.....	7
4.1 Architectural Style	7
4.2 Design Principles.....	7
4.3 Component Topology	7
5. Multi-Agent Pipeline Architecture.....	9
5.1 Pipeline Overview.....	9
5.2 Agent Specifications	9
5.3 LangGraph State Graph Definition	10
6. Technology Stack.....	11
6.1 Backend	11
6.2 Frontend	11
6.3 AI and Inference Layer	12
7. Shared State Schema.....	13
7.1 FactCheckState TypedDict	13
7.2 ClaimResult TypedDict.....	13
7.3 PipelineStatus Enumeration	14
8. API Design.....	15
8.1 REST Endpoint Catalogue	15
8.2 Server-Sent Events Protocol	15
9. Data Architecture	17
9.1 Database Overview.....	17
9.2 Database Tables	17
9.3 Data Retention	17
10. Ollama Deployment Configuration.....	18
10.1 Mode A — Local MacBook (Default)	18
10.2 Mode B — LAN-Connected Windows PC.....	18

10.3 Environment Configuration Reference.....	19
11. Non-Functional Requirements	20
12. Security and Privacy Considerations.....	21
12.1 Threat Model Scope	21
12.2 Privacy Controls.....	21
12.3 Known Limitations and Accepted Risks	21
13. Risk Register	22
14. Architecture Decision Records	23
ADR-01: LangGraph as the Agent Orchestration Framework.....	23
ADR-02: Qwen 2.5 3B as the Local Language Model.....	23
ADR-03: DuckDuckGo Search as the Evidence Retrieval Backend.....	23
ADR-04: Schema-First Development via Frozen TypedDict.....	24
Appendix A — Glossary	25
Appendix B — References	25

1. Document Information

1.1 Purpose

This document describes the complete system architecture for FactCheck AI, an autonomous multi-agent fact-checking application developed as a Final Year Project for the BSc (Hons) Software Engineering programme at ICBT Campus. It serves as the primary technical reference for all design decisions made during Phase 1 (Weeks 1–2) and as the authoritative contract for all subsequent development phases.

1.2 Intended Audience

- **Academic Supervisors:** Reference for assessment of architectural rigour and completeness.
- **Developer (Author):** Working reference throughout the 14-week implementation cycle.
- **Project Examiners:** Technical foundation for the live demonstration and viva.

1.3 Document Revision History

Version	Date	Author	Description of Change
1.0	May 2026	Hirushikanth Vidyanandan	Initial release — Phase 1 architecture definition

1.4 Related Documents

- Final Year Project Proposal — FactCheck AI (BSc_SE_CIS_6002_20305792)
- Phase 1 Implementation Plan (separate document)
- LangGraph Official Documentation — <https://langchain-ai.github.io/langgraph/>
- Qwen 2.5 Technical Report — arXiv:2412.15115

2. Executive Summary

FactCheck AI is a locally deployed, conversational fact-verification system that decomposes the complex task of claim verification into a coordinated pipeline of five specialised AI agents. Unlike existing cloud-based fact-checking tools, the system runs entirely on consumer hardware, requires no paid API subscriptions, and ensures complete user privacy by transmitting no data to third-party servers.

The system accepts natural-language input — a statement heard in a video, read on social media, or encountered in daily discourse — and produces a structured verdict for each constituent factual claim. Verdicts are labelled SUPPORTED, REFUTED, or INSUFFICIENT EVIDENCE, accompanied by a confidence score, a plain-English explanation, and cited source URLs retrieved from the live web. Following the initial analysis, users may continue the session with follow-up questions, which are answered by a dialogue agent grounded entirely in the evidence already retrieved.

The architecture centres on the LangGraph supervisor multi-agent pattern, with all agent coordination managed through a single shared TypedDict state object. The AI reasoning layer is provided by Qwen 2.5 3B, served locally through Ollama. The backend is implemented in Python using FastAPI, and the frontend is a React/TypeScript single-page application. Real-time pipeline progress is streamed to the browser via Server-Sent Events.

Key Architecture Principle: *Every verdict produced by the system must be traceable to a retrieved web source. The LLM is explicitly prohibited from generating verdicts from parametric memory alone, ensuring that all conclusions are evidence-grounded and externally verifiable.*

3. System Overview

3.1 Problem Context

The proliferation of misinformation on digital platforms has created a pressing need for accessible, automated fact-checking tools. Existing solutions suffer from four principal deficiencies: (1) they return binary true/false labels without supporting explanation; (2) they treat verification as a single-shot interaction with no conversational follow-up; (3) they transmit user queries to external cloud servers, raising privacy concerns; and (4) they do not differentiate the epistemic strength of claims supported by peer-reviewed literature from those supported only by blog posts.

3.2 System Goals

- **Evidence-grounded verdicts:** Every claim verdict is derived from live web evidence, not from LLM parametric memory.
- **Conversational continuity:** Users may ask follow-up questions within a session, with answers grounded in the evidence already retrieved.
- **Local operation:** The entire system runs on consumer hardware with no cloud API dependencies or ongoing costs.
- **Confidence transparency:** Each verdict carries a quantified confidence score and a plain-English justification.
- **Source traceability:** All cited sources are primary URLs that users can independently inspect.
- **Real-time feedback:** Pipeline progress is streamed to the frontend so users observe each processing step as it occurs.

3.3 Scope Boundaries

In Scope	Out of Scope
Natural-language claim input via web interface	Mobile native application
Five-agent LangGraph pipeline (Extractor, Verifier, Reporter, Dialogue, Orchestrator)	Multi-user concurrent sessions and authentication
DuckDuckGo-based web evidence retrieval	Integration with PubMed, academic databases, or proprietary fact-check repositories
Local Qwen 2.5 3B inference via Ollama	GPU cluster or distributed inference
Session persistence in SQLite	Cloud database or remote data storage
English-language input only	Multi-language support
Single-user local deployment	Adversarial input detection and hardening

4. Architecture Overview

4.1 Architectural Style

FactCheck AI employs a multi-agent pipeline architecture orchestrated by LangGraph. This style was selected over a monolithic service architecture for three reasons. First, it enables each subtask (claim extraction, evidence retrieval, verdict generation, report formatting, and dialogue) to be implemented, tested, and evaluated independently. Second, the LangGraph state graph provides deterministic control flow with explicit transitions between agents, which is essential for a production-grade information-verification system where the sequence of operations must be auditable. Third, the shared-state pattern allows all agents to operate on a single coherent data object without inter-process communication overhead.

The overall system follows a three-tier topology: a React/TypeScript frontend communicates with a FastAPI backend over HTTP/SSE, the backend hosts the LangGraph pipeline and exposes REST endpoints, and the pipeline communicates with a locally running Ollama instance (either on the same machine or on a LAN-connected host) for all LLM inference.

4.2 Design Principles

Principle	Description	Where Applied
Evidence-first reasoning	No LLM verdict is accepted without retrieved web evidence as supporting context.	Verifier Agent, Orchestrator validation
Separation of concerns	Each agent owns exactly one responsibility and has no knowledge of the internal workings of other agents.	All five agents
Schema-first development	The shared state TypedDict is defined and frozen in Phase 1 before any agent is implemented. Agents are contracts on this schema, not vice versa.	state.py — Phase 1 deliverable
Configuration over code	All environment-specific values (Ollama URL, model name, timeouts) live in .env and are injected via pydantic-settings. No hardcoded endpoints.	config.py, OllamaConfig
Fail-visible errors	Pipeline failures set error and status fields on the shared state and are surfaced to the frontend via SSE. There are no silent failures.	Orchestrator Agent, FastAPI error handlers
Privacy by architecture	No user input ever leaves the local machine boundary. DuckDuckGo queries are the only outbound network traffic.	Overall system design

4.3 Component Topology

The system is composed of four primary layers communicating as follows:

Layer	Components	Technology	Communication
Presentation Layer	Chat interface, real-time progress display, session history browser	React 18, TypeScript, Vite, Tailwind CSS	HTTP REST + Server-Sent Events
API Layer	REST endpoints, SSE streaming, request validation	FastAPI, Python 3.11+, Pydantic v2	HTTP over localhost:8000
Agent Pipeline Layer	Five specialised agents, LangGraph state graph, SQLite persistence	LangGraph, LangChain-Ollama, SQLAlchemy	In-process Python function calls
Inference Layer	Quantised LLM serving, model management	Ollama, Qwen 2.5 3B (Q4_K_M quantisation)	HTTP REST to localhost:11434 or LAN host

5. Multi-Agent Pipeline Architecture

5.1 Pipeline Overview

The fact-checking pipeline is implemented as a LangGraph StateGraph. The Orchestrator Agent acts as the supervisor, routing between agents based on the current pipeline state. Agents do not call each other directly; instead, each agent reads from and writes to the shared FactCheckState TypedDict, and the Orchestrator determines the next agent to invoke based on the updated state.

The nominal pipeline flow for a new fact-checking request is: Extractor → Verifier (once per extracted claim) → Reporter → [await user input] → Dialogue (for follow-up questions). The pipeline terminates when the user ends the session or when an unrecoverable error is encountered.

5.2 Agent Specifications

Agent	Responsibility	Primary Inputs	Primary Outputs	Key Constraints
Extractor Agent	Parses raw natural-language input into a list of atomic, self-contained, verifiable factual claims. Resolves pronoun references, disambiguates implicit subjects, and filters out subjective opinions that cannot be fact-checked.	raw_input (str)	extracted_claims (list[str])	Each output claim must be independently verifiable without reference to other claims. Subjective opinions and predictions must be excluded.
Verifier Agent	For each extracted claim, constructs targeted web search queries, retrieves evidence snippets via the DuckDuckGo API, and uses the local LLM to reason over the retrieved evidence to produce a verdict with a confidence score.	A single claim (str) + any prior context	ClaimResult (verdict, confidence, evidence, sources, reasoning, search_queries)	Verdict must be grounded in retrieved evidence. The LLM must not use parametric memory as the sole basis for any verdict. Invoked once per extracted claim.
Reporter Agent	Consolidates all per-claim ClaimResult objects into a single coherent, structured response formatted as a human-readable chat message. Computes an overall summary and formats source citations.	claim_results (list[ClaimResult])	final_report (str) — formatted markdown	Every cited URL must originate from a ClaimResult sources list. No new evidence or reasoning may be introduced at this stage.
Dialogue Agent	Maintains conversational continuity within a session. Answers follow-up questions using only the evidence and context	messages (conversation history) + claim_results + final_report	An assistant message appended to messages	Must not confabulate answers to questions that cannot be answered from the retrieved evidence.

Agent	Responsibility	Primary Inputs	Primary Outputs	Key Constraints
	retrieved during the initial pipeline run, without issuing new web searches.			Must explicitly state when a question requires new evidence outside the session context.
Orchestrator Agent	Manages the lifecycle of the entire pipeline using the LangGraph supervisor pattern. Routes between agents, manages shared state transitions, coordinates SSE progress streaming to the frontend, and handles error propagation.	FactCheckState (entire shared state)	Updated FactCheckState with current_agent and status fields reflecting current pipeline position	Must always update current_agent before delegating to another agent, enabling real-time frontend progress display. Must catch and surface all agent-level exceptions via the error field.

5.3 LangGraph State Graph Definition

The pipeline is implemented as a LangGraph StateGraph[FactCheckState]. The graph definition establishes the following edges:

- START → orchestrator: pipeline entry point on new fact-check request
- orchestrator → extractor: after validating non-empty raw_input
- extractor → verifier: after extracted_claims is populated with at least one claim
- verifier → verifier: conditional self-loop until all extracted claims have been processed
- verifier → reporter: after all claims have ClaimResult entries
- reporter → END: after final_report is populated (initial pipeline complete)
- dialogue → END: after each follow-up response is appended to messages

LangGraph Note: The Verifier Agent self-loop is implemented using a conditional edge that checks `len(claim_results) < len(extracted_claims)`. This allows the graph to process claims sequentially, which is necessary to stay within Ollama's single-request concurrency model on consumer hardware.

6. Technology Stack

6.1 Backend

Component	Technology	Version	Role
Runtime	Python	3.11+	Primary backend language; required by LangGraph and LangChain
API Framework	FastAPI	0.115+	REST API server, SSE streaming endpoint, request/response validation
Agent Orchestration	LangGraph	0.2+	Multi-agent state graph, supervisor pattern, conditional routing
LLM Integration	LangChain-Ollama	0.2+	Typed interface to Ollama's /api/generate and /api/chat endpoints
Web Search	DuckDuckGo Search	7.0+	Free, no-auth web evidence retrieval; text snippet extraction
Data Persistence	SQLAlchemy + SQLite	2.0+	Session and conversation log storage; zero-config local database
Settings Management	Pydantic-Settings	2.0+	Typed .env file loading; OllamaConfig, AppSettings classes
Dependency Management	Poetry	1.8+	Deterministic dependency resolution, virtual environment management
HTTP Client	HTTPX	0.27+	Async HTTP for Ollama smoke tests and health checks
ASGI Server	Uvicorn	0.32+	ASGI server with hot-reload for development

6.2 Frontend

Component	Technology	Version	Role
Framework	React	18+	Component-based UI, concurrent rendering for SSE updates
Language	TypeScript	5+	Type-safe frontend; FactCheckState types shared as interface definitions
Build Tool	Vite	5+	Sub-second HMR development server; optimised production build
Styling	Tailwind CSS	3+	Utility-first styling; dark-mode-first design system
State Management	Zustand	4+	Lightweight client state for session history and pipeline status

Component	Technology	Version	Role
HTTP + SSE	Native Fetch API	—	REST calls to FastAPI; EventSource API for SSE pipeline streaming

6.3 AI and Inference Layer

Component	Technology	Version / Spec	Role
LLM Runtime	Ollama	0.3+	Local LLM serving engine; manages model quantisation and GPU/CPU offloading
Language Model	Qwen 2.5	3B parameters (Q4_K_M)	Primary reasoning model for claim extraction, verdict generation, and dialogue
Quantisation Format	GGUF (Q4_K_M)	—	4-bit quantisation; ~2.2 GB VRAM/RAM footprint; balances quality and speed on consumer hardware
Inference Mode	CPU or GPU	8 GB RAM / 8 GB VRAM minimum	Supports both CPU-only (MacBook) and GPU-accelerated (Windows PC with discrete GPU) operation

7. Shared State Schema

The shared state schema is the single most architecturally significant artefact produced in Phase 1. It defines the data contract between all five agents and is frozen before any agent implementation begins. Any change to this schema after Phase 2 requires explicit version-bumping and a review of all agents that read or write the affected field.

7.1 FactCheckState TypedDict

The primary shared state object passed through the LangGraph pipeline. Defined in `factcheck/state.py`.

Field	Type	Owner Agent	Description
raw_input	str	User Input	The original natural-language text submitted by the user for fact-checking.
extracted_claims	list[str]	Extractor	List of atomic, self-contained factual claims parsed from raw_input.
claim_results	list[ClaimResult]	Verifier	List of verification results, one entry per extracted claim. Populated incrementally.
final_report	str None	Reporter	Formatted markdown report consolidating all verdicts. None until Reporter completes.
messages	Annotated[list[BaseMessage], add_messages]	Dialogue + User	Full conversation history. The add_messages reducer appends rather than replaces, enabling concurrent safe writes.
current_agent	str	Orchestrator	Identifier of the currently active agent. Streamed to frontend via SSE to display real-time progress.
session_id	str	Orchestrator	UUID for the current session. Used as the primary key for SQLite persistence.
error	str None	All agents	Populated by any agent that encounters an unrecoverable error. None indicates clean state.
status	PipelineStatus	Orchestrator	Enumerated pipeline status: idle running done error.

7.2 ClaimResult TypedDict

Represents the complete verification result for a single extracted claim. Produced by the Verifier Agent and consumed by the Reporter and Dialogue Agents.

Field	Type	Description
claim	str	The exact claim text as extracted by the Extractor Agent.
verdict	Literal['SUPPORTED', 'REFUTED', 'INSUFFICIENT_EVIDENCE']	The LLM's verdict on the claim based on retrieved evidence.
confidence	float (0.0 – 1.0)	Normalised confidence score. 0.0 = no supporting evidence; 1.0 = unanimous, high-quality evidence. Derived from LLM reasoning chain.
evidence	list[str]	Text snippets retrieved from DuckDuckGo search results that were provided to the LLM as context for this verdict.
sources	list[str]	Source URLs corresponding to the evidence snippets. All URLs cited in the final_report must originate from this list.
reasoning	str	The LLM's chain-of-thought explanation connecting the evidence to the verdict. Preserved verbatim for transparency.
search_queries	list[str]	The exact search queries issued to DuckDuckGo for this claim. Retained for auditability and failure analysis.

7.3 PipelineStatus Enumeration

A string literal type constraining the status field of FactCheckState to exactly four values:

Value	Meaning
idle	Pipeline has been initialised but no request has been received yet. Initial state.
running	A fact-check request is actively being processed. The current_agent field identifies which agent is active.
done	The pipeline has completed successfully. final_report is populated and available.
error	An unrecoverable error occurred. The error field contains a descriptive message. The session may not be continued.

8. API Design

8.1 REST Endpoint Catalogue

The FastAPI backend exposes the following endpoints. All endpoints return JSON. Error responses follow RFC 7807 Problem Details format.

Method	Path	Request Body	Response	Description
POST	/api/sessions	{ input: string }	{ session_id, status }	Creates a new fact-check session and initiates the agent pipeline.
GET	/api/sessions/{id}/stream	—	SSE event stream	Server-Sent Events stream for real-time pipeline progress. See §8.2.
GET	/api/sessions/{id}	—	FactCheckSession	Returns the complete session state including final_report and claim_results.
POST	/api/sessions/{id}/messages	{ message: string }	{ message_id }	Submits a follow-up question to the Dialogue Agent for an existing session.
GET	/api/sessions	—	list[SessionSummary]	Returns a paginated list of all past sessions stored in SQLite.
DELETE	/api/sessions/{id}	—	{ deleted: true }	Permanently deletes a session and all associated conversation history.
GET	/api/health	—	{ status, ollama_reachable, model_loaded }	System health check. Verifies Ollama connectivity and model availability.

8.2 Server-Sent Events Protocol

The /api/sessions/{id}/stream endpoint emits a sequence of typed events as the pipeline progresses. Each event carries a JSON payload. The client subscribes using the native EventSource API and updates the UI accordingly.

Event Type	Payload Fields	Description
agent_start	{ agent: string, timestamp: ISO8601 }	Emitted when the Orchestrator activates a new agent.
claim_found	{ claim: string, index: int, total: int }	Emitted by the Extractor Agent for each claim identified.
search_query	{ query: string, claim_index: int }	Emitted by the Verifier Agent before each DuckDuckGo search.
verdict_ready	{ claim: string, verdict: string, confidence: float }	Emitted by the Verifier Agent when a single claim verdict is complete.
report_ready	{ final_report: string }	Emitted by the Reporter Agent when the full report is assembled.
pipeline_done	{ session_id: string, duration_seconds: float }	Emitted by the Orchestrator on successful pipeline completion.
pipeline_error	{ error: string, agent: string }	Emitted by the Orchestrator on unrecoverable agent failure.
dialogue_reply	{ message: string }	Emitted by the Dialogue Agent for each follow-up response.

9. Data Architecture

9.1 Database Overview

All persistent data is stored in a single SQLite database located at `data/factcheck.db` within the backend directory. SQLite was chosen over a client-server database for three reasons: it requires zero configuration, it is bundled with Python's standard library, and it is entirely contained within the local filesystem, consistent with the system's privacy-by-architecture principle. The SQLAlchemy ORM provides a type-safe interface and supports future migration to PostgreSQL without application-layer changes.

9.2 Database Tables

Table	Primary Key	Key Columns	Description
sessions	session_id (UUID)	raw_input, status, created_at, completed_at, duration_seconds	One row per fact-check session. Stores the original user input and overall pipeline outcome.
claims	claim_id (UUID)	session_id (FK), claim_text, verdict, confidence, reasoning, created_at	One row per extracted claim. References its parent session. Stores the full ClaimResult.
evidence_items	evidence_id (UUID)	claim_id (FK), snippet_text, source_url, search_query	One row per evidence snippet retrieved for a claim. Normalised for efficient retrieval during dialogue.
messages	message_id (UUID)	session_id (FK), role (user assistant), content, created_at	Full conversation history for a session, including both the initial report and all follow-up dialogue turns.

9.3 Data Retention

All data is stored locally and is never transmitted to external services. Session data persists indefinitely until explicitly deleted by the user through the `DELETE /api/sessions/{id}` endpoint or by manually removing the SQLite database file. No telemetry, analytics, or logging data is sent outside the local machine.

10. Ollama Deployment Configuration

The system supports two Ollama deployment modes, selectable via a single environment variable. This design allows the developer to leverage any available hardware (MacBook CPU inference or a more powerful Windows PC with a dedicated GPU) without any code changes.

10.1 Mode A — Local MacBook (Default)

Ollama runs directly on the development MacBook. Inference is performed on CPU with Apple Silicon Metal acceleration where available. This mode is the default and requires no network configuration.

Parameter	Value
OLLAMA_BASE_URL	http://localhost:11434
Inference Type	CPU (Apple Silicon Metal acceleration if available)
Expected Latency	15 – 45 seconds per full pipeline run (8-claim input on M-series chip)
RAM Requirement	Minimum 8 GB system RAM; 16 GB recommended for comfortable concurrent operation
Model Path	~/ollama/models/qwen2.5:3b
Setup Command	ollama pull qwen2.5:3b && ollama serve

10.2 Mode B — LAN-Connected Windows PC

Ollama runs on a Windows PC connected to the same local network as the development MacBook. The PC must be configured to bind Ollama to all network interfaces, allowing the MacBook to connect over the LAN. This mode is recommended when the Windows PC has a discrete GPU with 8+ GB VRAM, as inference speed will be significantly faster.

Parameter	Value
OLLAMA_BASE_URL	http://<windows-pc-lan-ip>:11434
Windows Environment Var	OLLAMA_HOST=0.0.0.0 (set as a system environment variable before starting Ollama)
Inference Type	GPU-accelerated if a CUDA-compatible or ROCm-compatible GPU is present; CPU fallback otherwise
Expected Latency	3 – 8 seconds per full pipeline run (GPU with 8 GB VRAM; e.g. RTX 3060)
LAN IP Stability	Set a DHCP reservation in the router for the Windows PC's MAC address, or configure a static IP, to prevent the LAN IP from changing between sessions
Firewall	Windows Defender Firewall must permit inbound TCP connections on port 11434 from the MacBook's IP

Parameter	Value
PowerShell Setup	[Environment]::SetEnvironmentVariable('OLLAMA_HOST','0.0.0.0','Machine') then restart Ollama

10.3 Environment Configuration Reference

The following environment variables are defined in backend/.env (gitignored) and documented in backend/.env.example (committed):

Variable	Default	Description
OLLAMA_BASE_URL	http://localhost:11434	Base URL of the Ollama server. Change to LAN address for Mode B.
OLLAMA_MODEL	qwen2.5:3b	Model identifier as registered in Ollama. Must match the name used in ollama pull.
OLLAMA_TEMPERATURE	0.0	LLM temperature for verdict generation. 0.0 = deterministic; preferred for reproducibility.
OLLAMA_TIMEOUT	120	HTTP timeout in seconds for Ollama requests. Covers cold-start model load time on first inference.
OLLAMA_MAX_RETRIES	3	Number of retry attempts on Ollama API failures before marking the pipeline as errored.
DEBUG	false	Enables FastAPI debug mode and verbose LangGraph tracing logs.

11. Non-Functional Requirements

Category	Requirement	Target Metric	Measurement Method
Performance	Full pipeline execution time for a 3-claim input on CPU inference	≤ 3 minutes	Timed execution in evaluation Phase 8
Performance	Full pipeline execution time for a 3-claim input on GPU inference	≤ 45 seconds	Timed execution in evaluation Phase 8
Accuracy	Overall verdict accuracy on 30-claim evaluation dataset	$\geq 70\%$	Manual label comparison in Phase 8 evaluation
Accuracy	SUPPORTED class precision on evaluation dataset	$\geq 75\%$	Per-class precision metric in Phase 8 evaluation
Accuracy	REFUTED class precision on evaluation dataset	$\geq 70\%$	Per-class precision metric in Phase 8 evaluation
Reliability	Pipeline completion rate (no crash or timeout)	$\geq 90\%$ on standard test inputs	Automated test suite in Phase 8
Usability	Time for a non-technical user to complete a fact-check session	≤ 5 minutes end-to-end	User observation during live demo
Privacy	User query data transmitted to external services	Zero — all queries stay local	Network traffic monitoring
Availability	System uptime during development and demonstration	Stateless restart in < 10 seconds	Cold-start timing test
Portability	Operating system support	macOS (development), Windows (Ollama Mode B)	Manual verification
Maintainability	Cyclomatic complexity per agent module	≤ 15 per function	Radon static analysis in CI

12. Security and Privacy Considerations

12.1 Threat Model Scope

As a single-user local application, the system's threat model is narrower than a multi-user cloud deployment. The primary concerns are (a) inadvertent data exfiltration, (b) prompt injection via adversarially crafted input claims, and (c) web content poisoning where DuckDuckGo returns deliberately misleading or malicious pages.

12.2 Privacy Controls

Control	Mechanism	Threat Mitigated
No cloud LLM API calls	All inference is performed by locally served Ollama. No API keys for OpenAI, Anthropic, or any cloud provider are used or required.	User query content exposure to third-party AI providers
Local-only data persistence	SQLite database is stored in the local filesystem only. No cloud sync or remote backup.	Session history leakage to cloud storage
Minimal outbound network surface	The only outbound network traffic is DuckDuckGo search queries. No telemetry, no analytics, no third-party tracking libraries in the frontend.	Broad data exfiltration
DuckDuckGo — no account required	Search queries require no authentication and are not associated with a user account.	Search history linkability to user identity

12.3 Known Limitations and Accepted Risks

- **Prompt injection:** Adversarially crafted input (e.g., claims containing instruction-like text designed to manipulate the LLM's verdict) is not specifically defended against in the current scope. This is acknowledged as a limitation in §10 of the project proposal.
- **DuckDuckGo result quality:** The system accepts DuckDuckGo results at face value. Deliberately SEO-manipulated or misinformation-heavy pages may influence verdicts. Mitigation is partially provided by the multi-source evidence requirement.
- **LAN Mode B security:** In Mode B, the Ollama API is exposed on the local network without authentication. This is acceptable in a trusted home LAN context but would require an authentication layer in any shared network environment.

13. Risk Register

ID	Risk Description	Likelihood	Impact	Severity	Mitigation Strategy
R-01	Qwen 2.5 3B produces incorrect claim extractions for complex compound sentences, leading to incorrect verdict scope.	Medium	High	High	Implement multi-turn extraction with a verification pass. Document failure cases in Phase 8 evaluation.
R-02	DuckDuckGo rate-limiting causes evidence retrieval failures for pipelines with many claims.	Medium	Medium	Medium	Implement exponential backoff with configurable retry count (OLLAMA_MAX_RETRIES). Cache search results within a session.
R-03	LAN IP address of Windows PC (Mode B) changes between development sessions due to DHCP reassignment, breaking connectivity.	Low	Low	Low	Configure DHCP reservation in router. Document static IP setup in README. OLLAMA_BASE_URL is a single-line change in .env.
R-04	LLM inference latency on CPU-only MacBook exceeds acceptable threshold for a 5+ claim input, degrading user experience.	High	Medium	High	Implement claim-count warning in frontend. Evaluate Qwen 2.5 1B as a faster fallback. Document expected latency in user guide.
R-05	LangGraph StateGraph introduces difficult-to-debug state corruption when an agent raises an unhandled exception mid-pipeline.	Low	High	Medium	All agents wrapped in try/except. Errors populate state.error and trigger Orchestrator error handler. State is immutable between agent invocations.
R-06	SQLite concurrent write lock causes failures if the user submits a follow-up question before the pipeline has fully committed the session to the database.	Low	Low	Low	Pipeline writes to SQLite only on final completion. SSE status events inform frontend when writes are safe. Dialogue blocked until status == done.
R-07	Qwen 2.5 3B generates hallucinated source URLs in the reasoning field that are not present in the retrieved evidence snippets.	Medium	High	High	Reporter Agent validates all cited URLs against the sources lists in claim_results before including them in final_report. Invalid URLs are stripped with a warning.
R-08	14-week timeline is insufficient to complete all nine phases with the required evaluation and documentation quality.	Low	High	High	Modular architecture allows Phase 7 (frontend) to be simplified to a functional prototype if time pressure requires. Core pipeline (Phases 2–6) is the irreducible minimum.

14. Architecture Decision Records

Architecture Decision Records (ADRs) document the key technical decisions made during Phase 1 and the reasoning behind each choice. These records are intended to remain stable throughout the project and serve as justification for the choices made in the event of examiner questions.

ADR-01: LangGraph as the Agent Orchestration Framework

Attribute	Detail
Decision	Use LangGraph (LangChain's agent orchestration library) for all multi-agent coordination.
Alternatives Considered	CrewAI, AutoGen, custom Python pipeline with explicit function calls.
Rationale	LangGraph provides a deterministic state graph with explicit edge definitions, which is essential for an information-verification system where the sequence of operations must be auditable. Unlike CrewAI and AutoGen, LangGraph does not abstract away the agent-to-agent communication model, giving full control over state transitions. The supervisor pattern natively supports the Orchestrator-as-router design described in the proposal.
Consequences	The pipeline is defined as a compiled StateGraph, which requires the shared state schema to be defined before any agent is implemented. This is a net positive: it enforces schema-first development.

ADR-02: Qwen 2.5 3B as the Local Language Model

Attribute	Detail
Decision	Use Qwen 2.5 3B (Q4_K_M quantisation) as the sole LLM for all agent reasoning.
Alternatives Considered	Llama 3.2 3B, Phi-3 Mini 3.8B, Mistral 7B, Gemma 2 2B.
Rationale	Qwen 2.5 3B achieves stronger reasoning benchmarks than Llama 3.2 3B and Gemma 2 2B at equivalent parameter counts, particularly on instruction-following and information extraction tasks. At Q4_K_M quantisation it requires approximately 2.2 GB of VRAM/RAM, comfortably fitting within the 8 GB minimum hardware specification. Mistral 7B would offer higher quality but doubles the RAM requirement and triples inference latency on CPU.
Consequences	A fixed model version is used throughout the project for reproducibility. If quality is insufficient during Phase 8 evaluation, the model can be swapped to qwen2.5:7b without any code changes — only the OLLAMA_MODEL environment variable needs updating.

ADR-03: DuckDuckGo Search as the Evidence Retrieval Backend

Attribute	Detail
Decision	Use the duckduckgo-search Python library for all web evidence retrieval.

Attribute	Detail
Alternatives Considered	Tavily Search API, SerpAPI, Google Custom Search API, Bing Web Search API.
Rationale	DuckDuckGo is the only free, no-authentication, no-rate-limit (for moderate use) web search option available. Tavily offers superior result quality and an academic tier, but its API key requirement would introduce an external dependency and potential cost. Google and Bing APIs both require registration and have free-tier limits insufficient for evaluation-scale testing. The proposal's constraint of zero running costs makes DuckDuckGo the only viable choice.
Consequences	Result quality will be lower than Tavily or Google, particularly for obscure scientific claims. This limitation is acknowledged in the proposal's Limitations section and factored into the $\geq 70\%$ target accuracy.

ADR-04: Schema-First Development via Frozen TypedDict

Attribute	Detail
Decision	Define the complete FactCheckState and ClaimResult TypedDicts in Phase 1 and treat them as frozen contracts for all subsequent development phases.
Alternatives Considered	Define state schema incrementally as each agent is implemented; use a Pydantic BaseModel instead of TypedDict.
Rationale	LangGraph's StateGraph requires the state type to be passed at graph construction time. Defining it fully in Phase 1 prevents schema drift between agents — a failure mode observed in rapid-prototyping approaches where each developer adds fields without considering downstream consumers. TypedDict is preferred over Pydantic BaseModel for LangGraph state because it has lower overhead and is the pattern used in all official LangGraph documentation and examples.
Consequences	Any field added after Phase 2 begins requires a documented schema change with a review of all agents that touch the affected field. This adds a small overhead but prevents silent breakage.

Appendix A — Glossary

Term	Definition
Agent	A software component within the LangGraph pipeline that performs a single, well-defined subtask (e.g. claim extraction or verdict generation) by invoking one or more LLM calls and/or external tool calls.
Atomic Claim	A single factual statement that can be verified independently, contains no ambiguous pronoun references, and is not a subjective opinion or a prediction.
GGUF	GPT-Generated Unified Format — a binary file format for storing quantised LLM weights, used by Ollama. Q4_K_M denotes 4-bit quantisation with K-means clustering and medium quality.
LangGraph	An open-source framework by LangChain for building stateful, multi-agent AI workflows as directed graphs where nodes are agent invocations and edges are state transitions.
Ollama	An open-source tool for serving quantised large language models locally on consumer hardware. Provides an OpenAI-compatible HTTP API.
Parametric Memory	Knowledge encoded in an LLM's weights during training, as opposed to knowledge retrieved at inference time from an external source. The system explicitly prohibits using parametric memory as the sole basis for verdicts.
RAG	Retrieval-Augmented Generation — the pattern of augmenting an LLM's context window with documents retrieved from an external source before generating a response, to ground the output in real evidence.
SAFE	Search-Augmented Factuality Evaluator — a methodology from Wei et al. (Google DeepMind, 2024) that grounds factuality evaluation in web search results rather than LLM parametric knowledge.
SSE	Server-Sent Events — an HTTP-based protocol for unidirectional server-to-client streaming. Used by FastAPI to push real-time pipeline progress events to the React frontend.
StateGraph	The LangGraph construct that defines the agent pipeline as a directed graph. Each node is an agent function; edges are routing conditions evaluated after each node execution.
TypedDict	A Python typing construct that defines a dictionary with a fixed set of typed keys. Used as the LangGraph shared state type to provide IDE-level type safety without runtime overhead.
Verdict	One of three string labels produced by the Verifier Agent for each claim: SUPPORTED (evidence corroborates the claim), REFUTED (evidence contradicts it), or INSUFFICIENT_EVIDENCE (available evidence is inconclusive).

Appendix B — References

- Metropolitansky, D. (2025). Claimify: Extracting high-quality claims from language model outputs. Microsoft Research.
- Wei, J. et al. (2024). Long-form factuality in large language models. arXiv:2403.18802. Google DeepMind.
- LangChain AI. (2026). LangGraph Documentation. <https://langchain-ai.github.io/langgraph/>
- Qwen et al. (2025). Qwen2.5 Technical Report. arXiv:2412.15115.
- Lewis, P. et al. (2021). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401.
- Ollama. (2026). Official Documentation. <https://ollama.com/docs>
- Thorne, J. et al. (2018). FEVER: A Large-Scale Dataset for Fact Extraction and Verification. ACL Anthology N18-1074.

End of Document · FactCheck AI System Architecture Design Document · Version 1.0 · May 2026